

Large-Scale and Multi-Structured Databases



Eleonora Sgorbini

Megan Maremmanni

Chiara Masiero

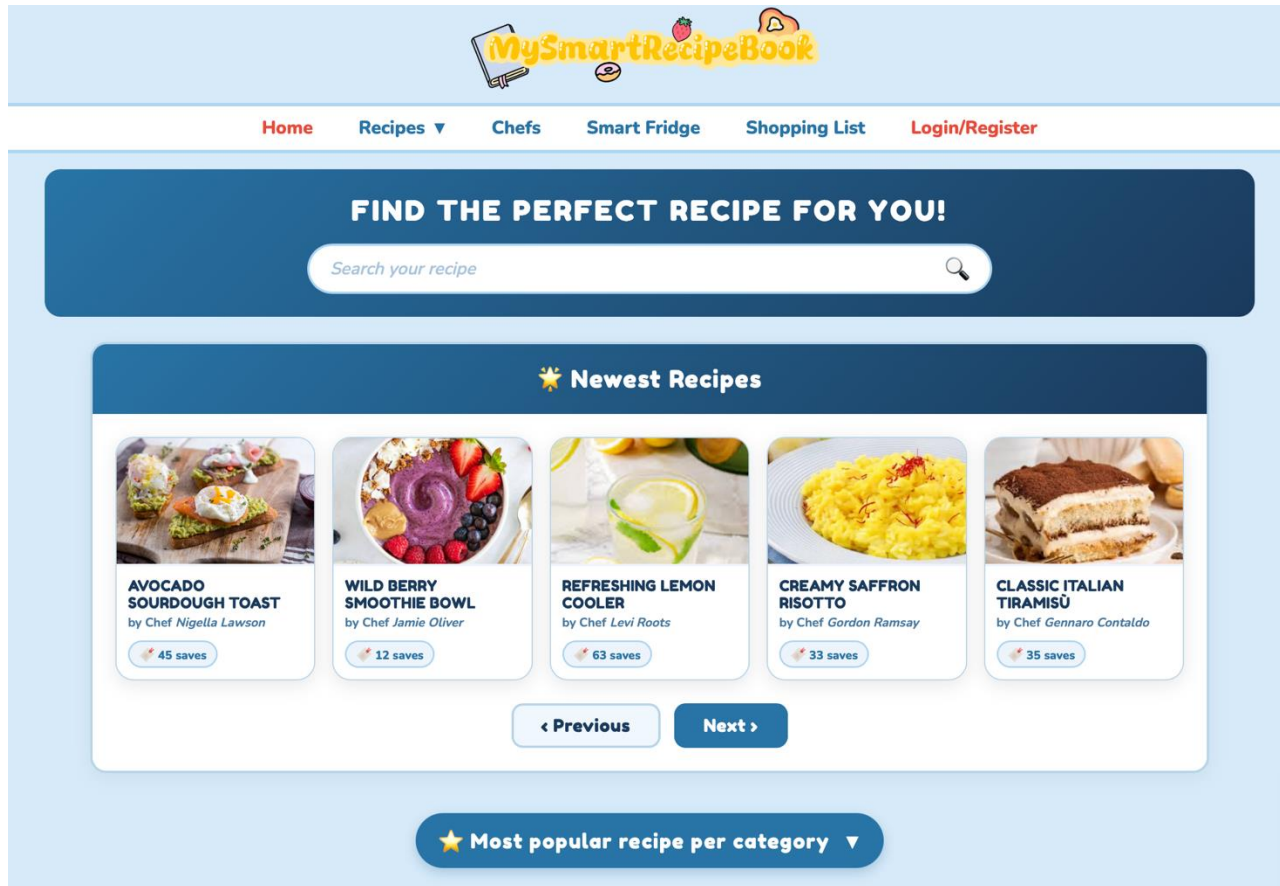
Application Highlights

MySmartRecipeBook is a recipe based web application designed to discover professional recipes and help you reducing food waste!

- **Advanced Search:** Users can browse through thousands of recipes using detailed filters such as category, or specific chefs, and save their favorite.
- **Smart Fridge:** This feature allows users to input available ingredients from their pantry to receive recipe recommendations, promoting culinary creativity and reducing food waste.
- **Smart Shopping List:** The application includes a portable digital shopping list that allows users to keep track of necessary ingredients while at the supermarket.
- **Application Analytics:** The platform provides insights into culinary trends, enabling the Admin to track monthly registrations, monitor the most popular chefs, and identify trending recipe categories.
- **Certified Content:** A dedicated administrative layer ensures a safe environment by verifying chefs' registration request and reviewing all submitted recipes before publication to maintain high-quality standards.

Actors and Mockups (i)

Unregistered User



The homepage is designed with a search bar for finding recipes via title or substring matching, and it immediately presents users with the newest recipes available in the application.

Actors and Mockups (ii)

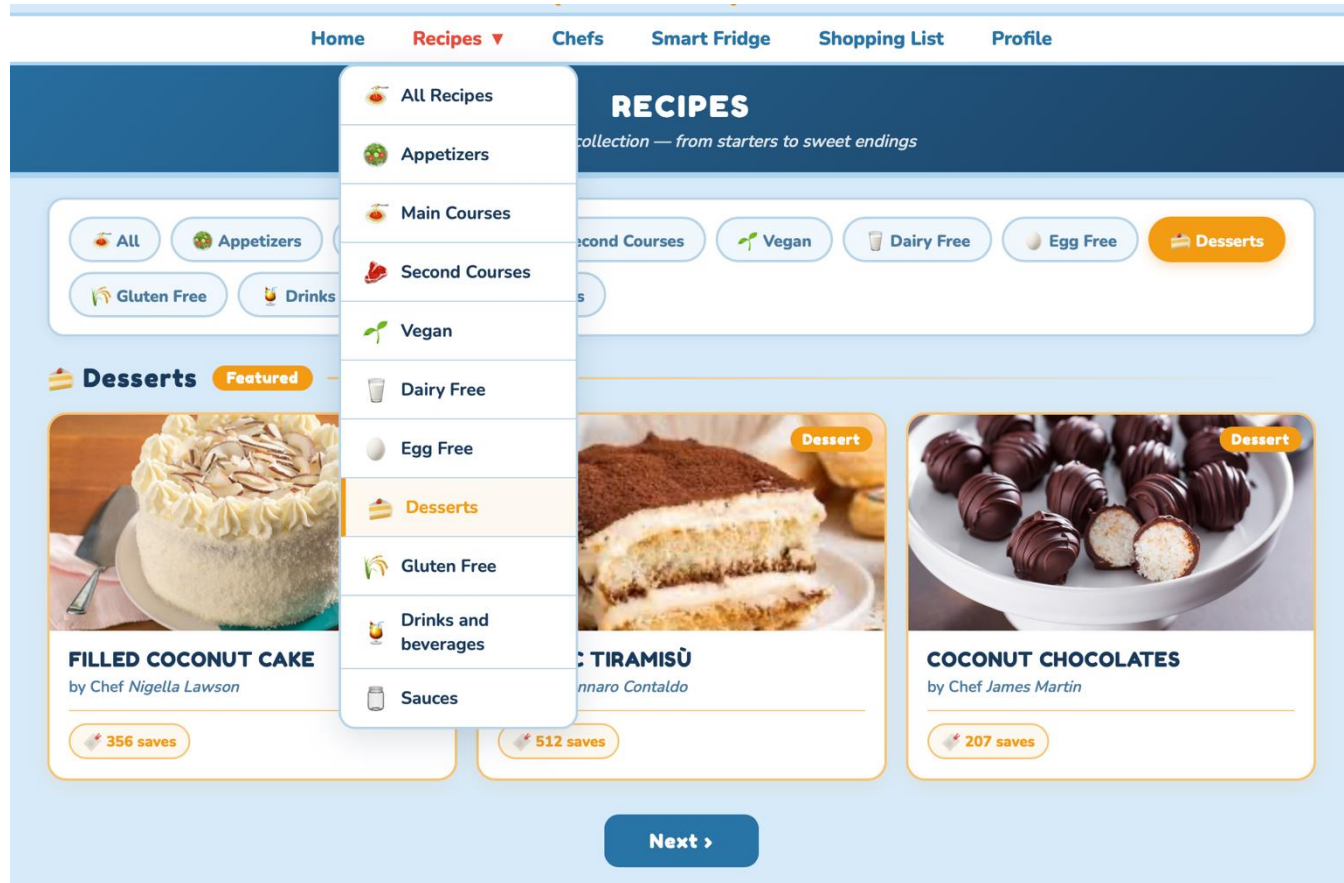
Unregistered User



By clicking the dedicated button on the homepage, users can instantly view the most popular recipe for each available category.

Actors and Mockups (iii)

Unregistered User



Any user can search recipes, browse categories and view complete recipe details before registering.

Actors and Mockups (iv)

Unregistered User

The screenshot displays the 'MySmartRecipeBook' website interface. At the top, the logo is accompanied by icons of a strawberry, a lemon, and a bowl. The navigation bar includes links for Home, Recipes, Chefs, Smart Fridge, Shopping List, and Profile. The 'Chefs' section is highlighted, featuring a search bar with the placeholder 'e.g. Allen'. Below the search bar, two chef profiles are shown: Rachel Allen with 6,099 saves and 193 recipes, and Darina Allen with 5,493 saves and 193 recipes. To the right, a detailed view of Nigella Lawson's recipes is shown, sorted by most recent. The list includes: Leek and Sausage Risotto (8 May 2025, 284 saves), Filled Coconut Cake (12 March 2024, 356 saves), Pumpkin and Amaretti Caprellacci (3 November 2023, 198 saves), Coconut Pancakes (15 July 2023, 123 saves), and Saffron Risotto (2 January 2023, 91 saves). Navigation buttons for 'Previous', 'Next', and 'See Similar' are visible at the bottom of the recipe list.

OUR CHEFS
Discover our chefs and their delicious recipes to get inspired

e.g. Allen

Rachel Allen
♥ 6,099 saves 📖 193 recipes

Darina Allen
♥ 5,493 saves 📖 193 recipes

Nigella Lawson

RECIPES – sorted by most recent

- LEEK AND SAUSAGE RISOTTO**
📅 8 May 2025 📖 284 saves
- FILLED COCONUT CAKE**
📅 12 March 2024 📖 356 saves
- PUMPKIN AND AMARETTI CAPPELLACCI**
📅 3 November 2023 📖 198 saves
- COCONUT PANCAKES**
📅 15 July 2023 📖 123 saves
- SAFFRON RISOTTO**
📅 2 January 2023 📖 91 saves

◀ Previous Next ▶

See Similar ▼

Unregistered users can browse chefs, searching them by surname and explore their published recipes.

Actors and Mockups (v)

Unregistered User



[Home](#) [Recipes ▾](#) [Chefs](#) [Smart Fridge](#) [Shopping List](#) [Login/Register](#)

FIND THE PERFECT RECIPE FOR YOU!

RECIPES



PUMPKIN ROLLS
by chef *Nadiya Hussain*
Saved 64 times



PUMPKIN CAKE
by chef *Nigella Lawson*
Saved 91 times

[< Previous](#) [Next >](#)

Recipe

FILLED COCONUT CAKE
by Chef Marcus Wareing



Filled Coconut Cake
by Chef Marcus Wareing · published 02/11/2019

[Desserts](#) [Average](#) [60 min](#)

[Save recipe](#)

You're craving a filled coconut cake with that Italian flair, huh? Well, this recipe's got you covered. It totally embraces the classic Italian vibe with its simple, moist goodness and a touch of class — perfect for those special moments. We're talking a cake with a yogurt-infused batter and shredded coconut that comes out so tender, giving you a base that's really flavorful. And the filling? A creamy mix of mascarpone and cream cheese. Seriously good. It adds this smooth, fresh burst that turns a simple cake into something truly special.

INGREDIENTS

Type 00 flour	2 ½ cups (300 g)
Sugar	1 ¼ cup (250 g)
Coconut yogurt	¾ cup (170 g)
Coconut flakes	1 ½ cup (135 g)
Sunflower seed oil	8.8 oz (250 g)
Milk	½ cup (125 g)
Eggs	1
Egg whites	1

PREPARATION

To prepare the filled coconut cake, first pour the coconut yogurt, milk, vegetable oil, and egg into a bowl. Add the egg white, lemon juice, and white wine vinegar. Mix until you obtain a homogeneous mixture. In another bowl, sift the flour and the baking soda, then add the sugar and shredded coconut. Mix well, then incorporate the dry ingredients into the liquid ones. Blend everything with a hand whisk until you get a smooth and homogeneous batter.

Butter two 8-inch springform pans, place a circle of parchment paper on the bottom, then flour the edges. Divide the batter equally between the pans. Place both pans on the lower rack of the oven and bake in a preheated static oven at 350°F for 35 minutes. During the last few minutes, check the cooking by doing the toothpick test. Once baked, remove from the oven and let cool completely.

Unregistered users can search recipes by keyword, browse matching results, and access the complete recipe view with all the detailed information.

Actors and Mockups (vi)

Log In and Register



Home Recipes ▾ Chefs Smart Fridge Shopping List Login

Login page

Foodie

Chef

Login as Foodie

Username

Password

Login

Don't have an account? [Register here](#)



Home Recipes ▾ Chefs Smart Fridge Shopping List Login

Register page

Foodie

Chef

Register as Foodie

Username Password

E-mail Name

Surname Birth date

Register

Already have an account? [Login here](#)

Actors and Mockups (vii)

Chef

Chef profile page

PERSONAL AREA
Welcome back Chef Nigella Lawson!

YOUR PERSONAL DATA
Modify

Chef: Nigella Lawson
E-mail: nigella.lawson@gmail.com
Password: *****
Birth date: 06/01/1960

WRITE A NEW RECIPE

PENDING RECIPES

 Saffron Risotto submitted 10-03-2025 Remove	 Lemon Tart submitted 15-03-2025 Remove	 Truffle Pasta submitted 18-03-2025 Remove	 Coconut Cake submitted 22-03-2025 Remove	 Avocado Toast submitted 24-03-2025 Remove
---	--	---	--	---

< Prev Next >

YOUR RECIPES – MOST RECENT

 Chocolate Muffin 12-01-2025 · 47 saves Delete
 Deviled Chicken 25-12-2024 · 83 saves Delete
 Summer Salad 03-11-2024 · 61 saves Delete

< Prev Next >

YOUR RECIPES – MOST POPULAR

 Coconut Pancakes 14-06-2023 · 312 saves Delete
 Risotto 02-03-2023 · 198 saves Delete
 Tiramisu Cake 20-09-2024 · 95 saves Delete

< Prev Next >

WARNING: Deleting your account will result in not being able to view your saved recipes, ShoppingList and SmartFridge anymore. Are you sure?

Delete Account

New recipe

NEW RECIPE

Insert recipe's title
Risotto allo zafferano

Insert recipe's presentation
Write a short presentation of your recipe...

Insert recipe's preparation
Step by step preparation...

Insert recipe's image
Click to upload or drag & drop

Category: Select category Difficulty: Select difficulty Preparation time: 30 min

Insert ingredients

Select Ingredient	100g
Select Ingredient	100g
Select Ingredient	100g

+ Save recipe Undo

✓

Recipe successfully added to pending recipes!

YOUR RECIPE

 **HONEY ROASTED PUMPKIN SOUP**
by chef Marcus Wareing

The Chef can manage personal info, create new recipes, monitor pending submissions and see their recipes.

Actors and Mockups (viii)

Admin



The Admin can handle pending recipes , chef registration requests and view the analytics by clicking the corresponding button.

Actors and Mockups (ix)

Foodie

Foodie profile page

PERSONAL AREA
Welcome back ele!

YOUR PERSONAL DATA
[Modify](#)

 **Name:** Eleonora
 **Surname:** Sgorbini
 **E-mail:** e.sgorbini@studenti.unipi.it
 **Password:** ••••••••
 **Birth date:** 01/01/1999

[View your ShoppingList](#)
[View your SmartFridge](#)

YOUR FAVOURITES
Filter by: [Select an option](#) ▼

 **FILLED COCONUT CAKE**
by chef Nigella Lawson [Remove](#)

 **CLASSIC TIRAMISÙ**
by chef Gennaro Contaldo [Remove](#)

[◀ Previous](#) [Next ▶](#)

WARNING: Deleting your account will result in not being able to view your saved recipes, ShoppingList and SmartFridge anymore. Are you sure?

[Delete Account](#)

The Foodie manages personal information, accesses Smart Fridge and Shopping List features, and keeps track of favourite recipes.

Actors and Mockups (x)

Foodie – See Similar

Recipes

Cream cheese1 ½ cup (375 g)

Icing sugar1.67 cups (200 g)

Coconut flakes (deco)0.67 cup (50 g)

Let the cake rest in the refrigerator for at least 2–3 hours before serving. Your filled coconut cake is ready!

See Similar ▾

**CLASSIC TIRAMISÙ**
by Chef Gennaro Contaldo

3 ingredients matched ✓
eggs sugar cream

**CHOCOLATE MUFFIN**
by Chef Nigella Lawson

4 ingredients matched ✓
flour eggs sugar milk

**LEMON TART**
by Chef Mary Berry

3 ingredients matched ✓
flour sugar cream

Chefs

< Previous

Next >

See Similar ▾

**Mary Berry**

**Gennaro Contaldo**

**Rachel Allen**

Actors and Mockups (xi)

Foodie – Smart Fridge

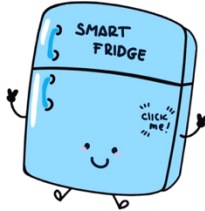


Home Recipes ▾ Chefs Smart Fridge Shopping List Profile



SMART FRIDGE

Add your ingredients and discover what you can cook!



SMART FRIDGE
CLICK ME!

Add at least 3 ingredients to your fridge:

eggs × flour ×

Add ingredient...

Add ingredients and get the suggestions

SUGGESTED RECIPES

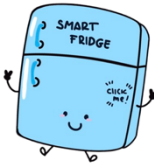
Add some ingredients and check your smart fridge!

Actors and Mockups (xii)

Foodie – Smart Fridge

SMART FRIDGE

Add your ingredients and discover what you can cook!



Add ingredients to your fridge:

+ lemon

+ cream

+ milk

Add ingredient...

Add ingredients and get the suggestions

YOUR INGREDIENTS

- ☐ eggs
- ☐ flour
- ☐ sugar
- ☐ coconut
- ☐ chocolate

SUGGESTED RECIPES



FILLED COCONUT CAKE
by Chef Nigella Lawson

4 ingredients matched ✓

coconut

flour

eggs

sugar



COCONUT PANCAKES
by Chef Monica Galletti

4 ingredients matched ✓

coconut

flour

eggs

sugar



COCONUT CHOCOLATES
by Chef James Martin

3 ingredients matched ✓

coconut

sugar

flour

< Previous

Next >

Actors and Mockups (xiii)

Foodie – Smart Shopping List



SHOPPING LIST

TO BUY ...

- eggs
- flour
- sugar
- coconut
- chocolate

Add new ingredients to your Shopping List

+ lemon

add

+ lemon zest

add

+ lemon juice

add

✨ Suggested Ingredients ▴

eggs pairs well with

butter	+ Add
milk	+ Add
baking powder	+ Add

flour pairs well with

yeast	+ Add
baking soda	+ Add
salt	+ Add

sugar pairs well with

vanilla extract	+ Add
brown sugar	+ Add
honey	+ Add

coconut pairs well with

coconut milk	+ Add
lime juice	+ Add
palm sugar	+ Add

chocolate pairs well with

heavy cream	+ Add
cocoa powder	+ Add
hazelnut paste	+ Add

Dataset Description

Sources (via Python Web-Scraping):

- . BBC GoodFood
- . AllRecipes
- . GialloZafferano

Volume:

- . 15,832 Recipes ≈ 53 MB
- . 5,000 Foodies
- . 93 Chefs ≈ 2.5 MB

Variety:

It is achieved by the fact that the web-service handles data retrieved from three different sources.

Velocity/Variability:

The system is highly dynamic, evolving rapidly through continuous data ingestion and user activity. This includes the regular insertion of new recipes, ongoing user registrations, recipe saving actions, dynamic Smart Fridge interactions, and real-time Shopping List updates.

Dataset Description

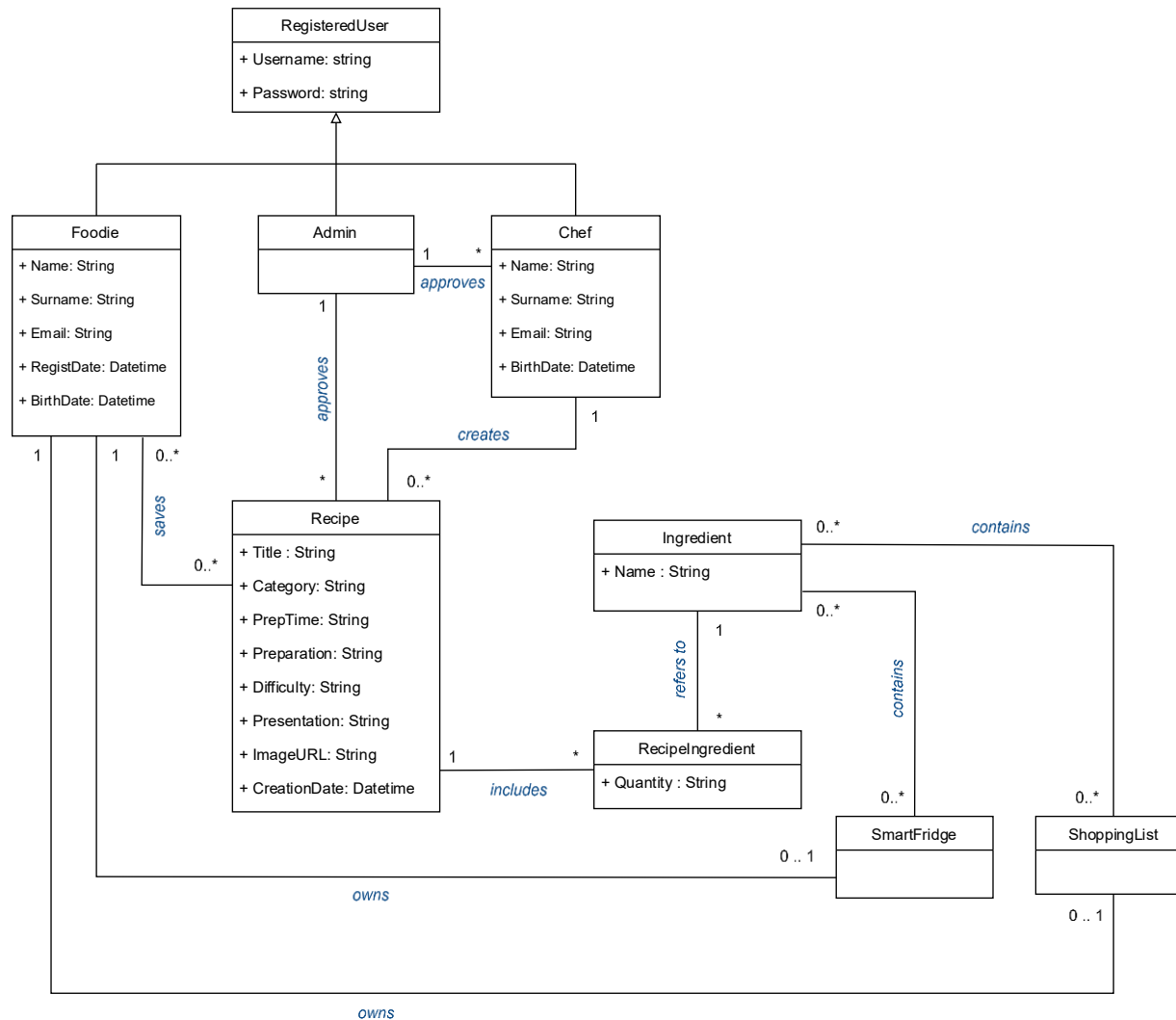
Recipes

- **Web Scraping Sources**
- **Deduplication:** Removed duplicate recipes.
- **Ingredient Normalization:** Normalized ingredients for consistent data within the application.
- **Text Cleaning:** Eliminated artifacts and redundant formatting characters.
- **Missing Data Generation:** Manually integrated or deduced missing attributes by re-consulting sources and cross-referencing web data.

Users

- **Foodie Profiles :** Generated using the Python *Faker* library to populate realistic personal attributes (names, emails, registration dates, birthdays).
- **Chef Profiles:** Real chefs were scraped from culinary websites to ensure authentic professional identities, while all other metadata was generated via Python *Faker* library.

UML Design Class Diagram



Application non-functional requirements

The **non functional-requirements** of our system are the following:

- The system must encrypt the user's password.
- The system must be developed by using an OOP language (Java).
- The system must be a responsive website application.
- The system must be able to handle high traffic.
- The system must implement a multi-database architecture using DocumentDB, GraphDB, and Key-Value DB.
- The system must expose all its functionalities by means of well-documented RESTful APIs, in accordance with the OpenAPI specification.

Document DB Design

Foodie

```
_id: ObjectId('69cea20c14f896db429b5a79')
username: "sierra.mitchell"
name: "Sierra"
surname: "Mitchell"
email: "sierra.mitchell@hotmail.com"
registration_date: 2024-11-12T00:00:00.000+00:00
password: "$2b$04$fJtPdwoGxDMSbOnsEE6RkeJX9gTvxd7AH322Bhh2dg1.Zb8psmvFi"
_class: "it.unipi.MySmartRecipeBook.model.Foodie"
saved_recipes: Array (173)
birthdate: 2006-07-25T00:00:00.000+00:00
```



```
▼ saved_recipes : Array (173)
  ▼ 0: Object
    id : "69cea24d14f896db429b8b29"
    title : "Crispy Salmon with Panko"
    category : "Main courses"
    difficulty : "Easy"
    image_url : "https://www.giallozafferano.com/images/344-34442/crispy-salmon-with-pa..."
    saving_date : 2026-06-02T22:00:00.000+00:00
    ▶ chef : Object
  ▶ 1: Object
  ▶ 2: Object
```

Document DB Design

Chef



```
_id: ObjectId('69ce825a14f896db429a146f')
username: "nadiya.hussain"
new_recipes: Array (15)
  ▶ old_recipes: Array (147)
  ▶ popular_recipes: Array (16)
tot_saves: 5206
tot_recipes: 162
status: "APPROVED"
name: "Nadiya"
surname: "Hussain"
email: "nadiya.hussain@yahoo.com"
password: "$2b$04$178iTz6zwQsFYR7r1eSuIu0rEqJZx9man0jMx7cR5lUzkfsGE.X/a"
birthdate: 1949-05-18T23:00:00.000+00:00
_class: "it.unipi.MySmartRecipeBook.model.Mongo.users.Chef"
▼ recipes_to_confirm: Array (1)
  ▼ 0: Object
    id: "6a2180330fc2e5349e11dc84"
    title: "Sweet Cherry Pie"
    image_url: "https://images.unsplash.com/photow=800&q=80"
    creation_date: 2026-06-04T13:40:03.537+00:00
```

Document DB Design

Chef



```
new_recipes : Array (15)
  ▼ 0: Object
    id : "69cea24c14f896db429b7422"
    title : "Vegetable vegan biriyani with carrot salad"
    image_url : "https://images.immediate.co.uk/production/volatile/sites/30/2020/08/re..."
    creation_date : 2026-02-20T23:53:00.124+00:00
    num_saves : 32
  ▶ 1: Object
  ▶ 2: Object
```



```
_id: ObjectId('69ce825a14f896db429a146f')
username : "nadiya.hussain"
▶ new_recipes : Array (15)
▼ old_recipes : Array (147)
  0: "69cea24c14f896db429b7b6a"
  1: "69cea24c14f896db429b81a9"
  2: "69cea24e14f896db429ba682"
```



```
popular_recipes : Array (16)
  ▼ 0: Object
    id : "69cea24e14f896db429b9e89"
    title : "Grilled Tandoori Chicken Thighs"
    image_url : "https://imagesvc.meredithcorp.io/v3..."
    creation_date : 2022-09-29T12:26:02.124+00:00
    num_saves : 45
  ▶ 1: Object
```

Document DB Design

Admin

```
_id: ObjectId('69cea60d14f896db429babe3')
_class: "it.unipi.MySmartRecipeBook.model.Admin"
password: "$2a$10$ZWl8y0ftWkI36aJQbvLHh.b.BqYYEVP6tvHHfgmBzMisbGma1jWC"
username: "admin"
▼ recipes_to_approve : Array (1)
  ▼ 0: Object
    ▼ chef : Object
      id: "69ce825a14f896db429a146d"
      name: "Nigella"
      surname: "Lawson"
      id: "6a2a7b133f6583321a0ffd1e"
      title: "Sweet Apple Pie"
      image_url: "https://images.unsplash.com/photow=800&q=80"
      creation_date: 2026-06-11T09:08:35.704+00:00
▼ chefs_to_approve : Array (1)
  ▼ 0: Object
    _id: ObjectId('6a2a7d783f6583321a0ffd1f')
    username: "garrett.graham"
    name: "Garrett"
    surname: "Graham"
```

Document DB Design

Recipe

```
_id: ObjectId('69cea24b14f896db429b6e0f')
title: "Easy Pineapple Upside-Down Cake"
image_url: "https://www.allrecipes.com/thmb/HV7Yth9XMLJD4nur5MsQwlnbdLA=/1500x0/fi..."
category: "Desserts"
difficulty: "Easy"
prep_time: "15 mins"
presentation: "Here's a quick and easy pineapple upside-down cake recipe using any br..."
▼ ingredients: Array (9)
  ▶ 0: Object
  ▶ 1: Object
  ▶ 2: Object
  ▼ 3: Object
    name: "pecans"
    quantity: "2 tablespoons"
  ▶ 4: Object
  ▶ 5: Object
  ▶ 6: Object
  ▶ 7: Object
  ▶ 8: Object
  preparation: "Preheat the oven to 350 degrees F (175 degrees C). Melt butter in a 9..."
▼ chef: Object
  id: "69ce825a14f896db429a146f"
  name: "Nadiya"
  surname: "Hussain"
creation_date: "2021-05-12T19:43:39.124"
_class: "it.unipi.MySmartRecipeBook.model.Mongo.RecipeMongo"
num_saves: 29
status: "APPROVED"
```

Relevant MongoDB queries

Chef Ranking using Bayesian Score

```
@Aggregation(pipeline = {
  "{ $match: { username: { $ne: 'admin' }, status: { $ne: 'pending' } } }",

  "{ $addFields: { " +
    "  tot_saves: { $ifNull: ['$tot_saves', 0] }, " +
    "  tot_recipes: { $ifNull: ['$tot_recipes', 0] } " +
    " } }",

  "{ $setWindowFields: { partitionBy: null, output: { " +
    "totalSavesGlobal: { $sum: '$tot_saves' }, " +
    "totalRecipesGlobal: { $sum: '$tot_recipes' } } } }",

  "{ $addFields: { C: { $cond: [ { $eq: ['$totalRecipesGlobal', 0] }, 0, " +
    " { $divide: ['$totalSavesGlobal', '$totalRecipesGlobal'] } ] } } }",

  "{ $addFields: { R: { $cond: [ { $eq: ['$tot_recipes', 0] }, 0, " +
    " { $divide: ['$tot_saves', '$tot_recipes'] } ] } } }",

  "{ $addFields: { score: { $add: [ " +
    " { $multiply: [ { $divide: ['$tot_recipes', { $add: ['$tot_recipes', 5] } ] }, '$R' ] }, " +
    " { $multiply: [ { $divide: [5, { $add: ['$tot_recipes', 5] } ] }, '$C' ] } " +
    " ] } } }",

  "{ $sort: { score: -1 } }",

  "{ $setWindowFields: { sortBy: { score: -1 }, output: { rank: { $rank: {} } } } }",
  "{ $limit: 5 }",
  "{ $project: { rank: 1, name: 1, surname: 1, score: 1 } }"
})
```

```
List<ChefRankAnalyticsDTO> chefBayesianRanking();
```

This query ranks chefs through a Bayesian score that combines their total number of saves and the total number of recipes, highlighting the most consistently appreciated profiles.

Relevant MongoDB queries

Monthly Foodie Registration Statistics

```
@Aggregation(pipeline = {
  "{ $match: { $expr: { $eq: [{ $year: '$registration_date' }, ?0] } } }",
  "{ $group: { " +
    "    _id: { $dateToString: { format: '%Y-%m', date: '$registration_date' } }, " +
    "    year: { $first: { $dateToString: { format: '%Y', date: '$registration_date' } } }, " +
    "    number: { $sum: 1 } " +
    "  } }",

  "{ $sort: { 'year': -1, 'number': -1 } }",

  "{ $group: { " +
    "    _id: '$year', " +
    "    totalRegisteredFoodies: { $sum: '$number' }, " +
    "    monthAnalyticsDTOList: { $push: { month: '$_id', totalFoodies: '$number' } } " +
    "  } }",

  "{ $project: { " +
    "    _id: 0, " +
    "    year: { $toInt: '$_id' }, " +
    "    totalRegisteredFoodies: 1, " +
    "    monthAnalyticsDTOList: 1 " +
    "  } }"
})

YearAnalyticsDTO getMonthlyFoodiesStats(int year);
```

Filters Foodie registrations by a target year to compute the overall annual growth and detailed monthly acquisition metrics

Relevant MongoDB queries

Category Trend Analysis

```
@Aggregation(pipeline = {
  "{ $match: { status: { $ne: 'pending' } } }",
  "{ $addFields: { " +
    "is_recent: { $gte: [ { $toDate: '$creation_date' }, ?0 ] }, " +
    "is_previous: { $and: [ " +
    "{ $lt: [ { $toDate: '$creation_date' }, ?0 ] }, " +
    "{ $gte: [ { $toDate: '$creation_date' }, ?1 ] } " +
    "] } " +
    "}" },",
  "{ $group: { " +
    "_id: '$category', " +
    "recentCount: { $sum: { $cond: [ '$is_recent', 1, 0 ] } }, " +
    "previousCount: { $sum: { $cond: [ '$is_previous', 1, 0 ] } } " +
    "}" },",
  "{ $addFields: { " +
    "growthRate: { $cond: [ " +
    "{ $gt: [ '$previousCount', 0 ] }, " +
    "{ $divide: [ { $subtract: [ '$recentCount', '$previousCount' ] }, '$previousCount' ] }, " +
    "null " +
    "] } " +
    "}" }"
})
```

Compares the number of recipes created in a recent period with a previous one, identifying which categories are growing or decreasing over time.

```
List<TrendAnalyticsDTO> findCategoryTrend(LocalDateTime recentDate, LocalDateTime previousDate);
```

Relevant MongoDB queries

Most Saved Recipes per Category

```
@Aggregation(pipeline = {
  "{ $match: { status: 'APPROVED' } }",
  "{ $sort: { num_saves: -1 } }",
  "{ $group: { " +
    "  _id: '$category', " +
    "  recipeId: { $first: '$_id' }, " +
    "  title: { $first: '$title' }, " +
    "  chef: { $first: '$chef' }, " +
    "  num_saves: { $first: '$num_saves' }, " +
    "  image_url: { $first: '$image_url' } " +
    "} }",
  "{ $project: { " +
    "  _id: '$recipeId', " +
    "  category: '$_id', " +
    "  title: 1, " +
    "  chef: 1, " +
    "  num_saves: 1, " +
    "  image_url: 1 " +
    "} }"
})
```

This query identifies the most saved approved recipe for each category, highlighting the recipes that users appreciate the most within every food category.

```
List<RecipeMongo> findMostSavedRecipePerCategory();
```

MongoDB Replica Set Configuration

Three replicas were implemented with the following configuration:

```
{ _id: 0, host: 'mongo_machine1:10.1.1.92', priority: 5 }  
{ _id: 1, host: 'mongo_machine2:10.1.1.91', priority: 2 }  
{ _id: 2, host: 'mongo_machine3:10.1.1.90', priority: 1 }
```

The priority parameters are chosen to handle the election process in the event of a primary node failure.

Write Concern and Read Preference

- **Write Concern: Write concern 1 (W1).** This ensures fast write operations and low latency, accepting a minimal risk of data loss in the rare event of a Primary failure immediately before the asynchronous replication to Secondaries occurs.
- **Read Preference: Nearest.** This effectively distributes the read load across the available infrastructure and minimizes query response times.

MongoDB Indexes

Recipes Collection

creation_date (Descending Index)

Descending index on the creation_date field was added to drastically optimize the retrieval of the newest recipes

Metric	Without index	With index
Documents returned	5	5
Execution time	73 ms	2 ms
Index keys examined	0	5
Documents examined	15,832	5

title (Text Index)

Users search via partial keywords or multi-word phrases (findByTitleContainingIgnoreCase).

Metric	Without index	With Text index
Documents returned	28	28
Execution time	58 ms	22 ms
Index keys examined	0	1010
Documents examined	15,832	1060

category (Standard Index)

Added to optimize category-based recipe browsing, one of the most frequent operation in the application.

Metric	Without index	With index
Documents returned	3011	3011
Execution time	96 ms	19 ms
Index keys examined	0	3011
Documents examined	15,832	3011

MongoDB Indexes

Chefs Collection

username (Unique Index)

Optimizes login, registration and profile retrieval and guarantees the uniqueness of the usernames.

surname (Standard Index) X Not retained

Evaluated for improving surname-based chef search. Considered not strictly necessary in the current version, it could become relevant as the collection grows.

Metric	Without index	With index
Documents returned	1	1
Execution time	0 ms	0 ms
Index keys examined	0	1
Documents examined	94	1

Foodies Collection

username (Unique Index)

Introduced for data integrity during registration.

- Ensures no two foodies can register with the same username.
- Useful for the most frequent operations such as login

Discussion on MongoDB Data Sharding

MongoDB

- **Foodies (Hashed on username):** Uniform data distribution. Optimizes the authentication process by enabling highly efficient, direct lookups without broadcast queries.
- **Chefs (Future: Hashed on _id):** Currently unsharded due to low volume. The future strategy prioritizes high-frequency profile reads over a slight overhead during login.
- **Recipes (Hashed on _id):** Optimizes the retrieval of recipe details (dominant operation).

Redis (Smart Fridge & Shopping List)

- **Current Architecture:** Sentinel Master-Slave. The workload is heavily read-bound, making a Cluster/Sharding approach unnecessary at this stage.
- **Future Development:** If write operations increase, the system will migrate to a Redis Cluster. It will adopt a key hashing strategy using *Hash Tags* to ensure all data for a single user is routed to the same shard.

Key-value DB Design

Keys Naming Conventions across the different features

Shopping List

Foodie: <username> : shoppingList

Smart Fridge

Foodie: <username> : smartFridge : ingredients

Foodie: <username> : smartFridge : suggestions

Allowed Ingredients

Allowed_ingredients

Key-value DB Design

Allowed Ingredients

Centralized in-memory set that validates ingredients across the whole platform, both in Foodies' lists and Chefs' recipe creation.

```
SMEMBERS Allowed_ingredients  
  "pork belly"  
  "mascarpone cheese"  
  "brown sugar"  
  "..."
```

Shopping List

Set of ingredients a Foodie plans to buy. Every input is normalized to lowercase before storage.

```
SMEMBERS Foodie:sierra.mitchell:shoppingList  
  "butter"  
  "bread"  
  "spaghetti pasta"  
  "avocados"
```

Key-value DB Design

Smart Fridge – Ingredients

Set of ingredients a Foodie has at home (same lowercase normalization as the Shopping List). A recipe is suggested when at least 3 of them match.

SMEMBERS

```
Foodie:sierra.mitchell:smartFridge:ingredients  
  "bread"  
  "butter"  
  "avocados"
```

Smart Fridge – Suggestions

Computed via the Graph DB, then cached in Redis as a sorted JSON list to reduce latency.

```
GET Foodie:sierra.mitchell:smartFridge:suggestions  
[ {  
  "id" : "69cea24e14f896db429ba7b7",  
  "title" : "Best Chicken Panini",  
  "chef" : "Gordon Ramsay",  
  "matchCount" : 3,  
  "matchedIngredients" : [ "bread", "butter",  
    "avocados" ]  
}, { ... } ]
```

Redis Replica Set Configuration

To guarantee high availability and eliminate a critical Single Point of Failure (SPOF) for caching and session management, a **Master-Replica** architecture was implemented.

The data topology distributes:

- Single **Master** node on machine 10.1.1.92
- Two **Replica** nodes on machines 10.1.1.91 and 10.1.1.90

All nodes are operating on port 6379 and Sentinel on port 7004.

To ensure fault tolerance, **Redis Sentinel** is configured across the infrastructure, in order to constantly monitor the state of the nodes.

Relevant Redis queries

Ingredients Management

```
public Set<String> getAllAllowedIngredients() {  
    try (Jedis jedis = jedisSentinelPool.getResource()) {  
        return jedis.smembers(INGREDIENTS_REDIS_KEY);  
    }  
    catch (JedisException e) {  
        return new HashSet<>();  
    }  
}
```

Shopping List & Smart Fridge Management – add ingredient(s)

```
String key = REDIS_KEY_PREFIX + foodie.getUsername() + REDIS_KEY_FRIDGE_SUFFIX;  
  
if (!ingredients.isEmpty()) {  
    try (Jedis jedis = jedisSentinelPool.getResource()) {  
        jedis.sadd(key, ingredients.toArray(new String[0]));  
        jedis.del(key: REDIS_KEY_PREFIX + foodie.getUsername() + REDIS_KEY_RECIPES_SUFFIX);  
        jedis.expire(key, seconds: 86400*15);  
    }  
}
```

Relevant Redis queries

Shopping List & Smart Fridge Management – remove ingredient

```
if(ingredientService.isValidIngredient(ingredient)) {  
    String key = REDIS_KEY_PREFIX + foodie.getUsername() + REDIS_KEY_FRIDGE_SUFFIX;  
    try (Jedis jedis = jedisSentinelPool.getResource()) {  
        jedis.srem(key, ingredient);  
        jedis.expire(key, seconds: 86400*15);  
    }  
    updateCacheAfterRemoval(foodie.getUsername(), ingredient);  
}
```

Recipe Suggestions Caching

```
String cacheKey = REDIS_KEY_PREFIX + username + REDIS_KEY_RECIPES_SUFFIX;  
  
String json;  
try (Jedis jedis = jedisSentinelPool.getResource()) {  
    json = jedis.get(cacheKey);  
}
```

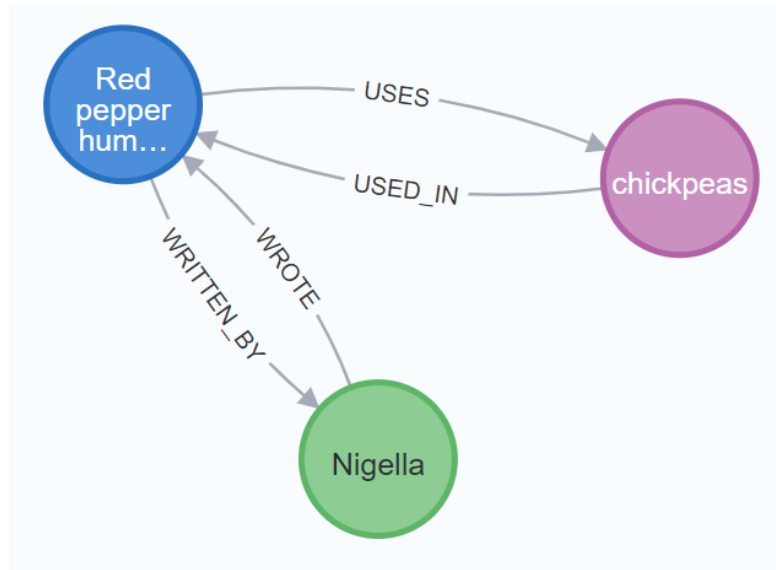
Graph DB Design

Nodes

Chef: authors of the recipes with basic information: *mongo_id*, name and surname.

Ingredient: ingredients used in recipes.

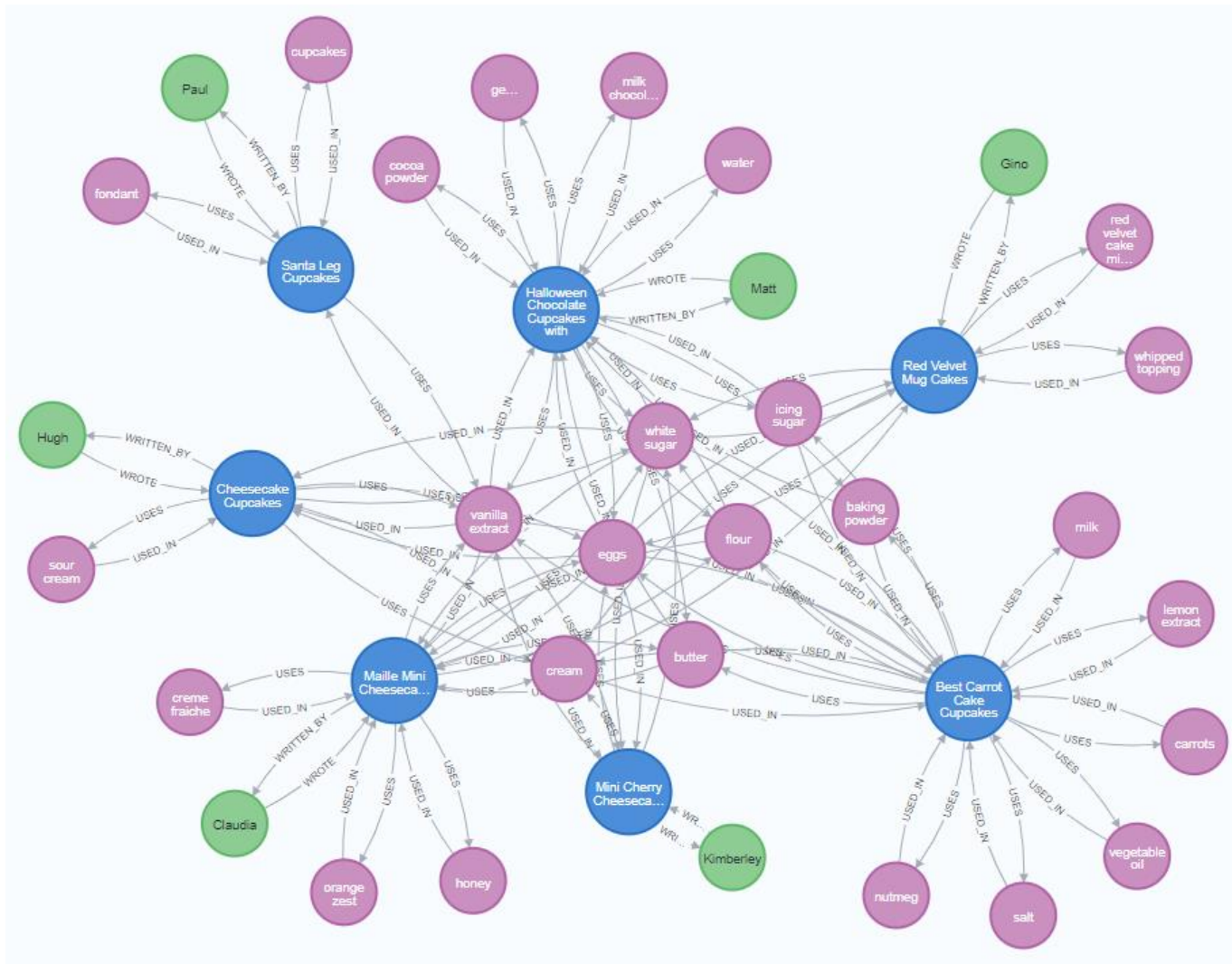
Recipe: recipes in the system. Each node contains: *mongo_id*, title, image URL and category.



Relations

- **:USED_IN:** connects an *Ingredient* to a *Recipe*, representing which ingredients are used in a given recipe.
- **:USES:** connects a recipe to an ingredient, is the inverse of **USED_IN**
- **:WRITTEN_BY:** connects a *Recipe* to a *Chef*, representing the author of the recipe.
- **:WROTE:** connects a *Chef* to a *Recipe*. Is the inverse navigation of **:WRITTEN_BY**

Graph DB Design



Relevant Neo4j queries

Similar Chefs by Shared Ingredients

```
@Query("MATCH (target:Chef {mongo_id: $targetMongoId})-[:WROTE]->(r1:Recipe)-[:USES]->(i:Ingredient) " +  
    "WITH target, i, collect(DISTINCT r1.category) AS targetCategories " +  
    "WHERE COUNT { (i)-[:USED_IN]->() } < 350 " +  
    "WITH DISTINCT target, i, targetCategories " +  
    "MATCH (i)-[:USED_IN]->(r2:Recipe)-[:WRITTEN_BY]->(other:Chef) " +  
    "WHERE target <> other AND r2.category IN targetCategories " +  
    "WITH other, count(DISTINCT i) AS sharedIngredientsScore " +  
    "ORDER BY sharedIngredientsScore DESC " +  
    "LIMIT 3 " +  
    "RETURN other.mongo_id AS id, " +  
    "        other.name AS name, " +  
    "        other.surname AS surname")
```

```
List<ChefInfoDTO> findSimilarChefs(@Param("targetMongoId") String targetMong
```

Identifies most similar chefs by analyzing shared specialized ingredients within matching recipe categories.

Relevant Neo4j queries

Suggested Complementary Ingredients

```
@Query("MATCH (target:Ingredient)-[:USED_IN]->(r:Recipe)-[:USES]->(other:Ingredient) " +  
    "WHERE target.name IN $ingredientList " +  
    "AND NOT other.name IN $ignoredIngredients " +  
    "WITH target.name AS originalIngredient, other.name AS suggestedIngredient, count(r) AS occurrences " +  
    "ORDER BY occurrences DESC " +  
    "RETURN originalIngredient, collect(suggestedIngredient)[0..3] AS suggestedIngredients")  
List<IngredientSuggestionDTO> findSuggestedIngredientsForList(  
    @Param("ingredientList") List<String> ingredientList,  
    @Param("ignoredIngredients") List<String> ignoredIngredients  
);
```

Recommends complementary ingredients by finding the ones most frequently paired with the user's current shopping list, filtering out the list itself and most common ingredients

Relevant Neo4j queries

Similar Recipes

```
@Query("MATCH (target:Recipe {mongo_id: $recipeId})-[:USES]->(i:Ingredient)-[:USED_IN]->(other:Recipe) " +
    "WHERE target <> other " +
    "WITH other, count(i) AS sharedIngredientsCount, collect(i.name) AS sharedIngredients " +
    "MATCH (other)-[:WRITTEN_BY]->(c:Chef) " +
    "RETURN other.mongo_id AS id, " +
    "        other.title AS title, " +
    "        other.imageUrl AS imageUrl, " +
    "        c.name AS chefName, " +
    "        c.surname AS chefSurname, " +
    "        c.mongo_id AS chefId, " +
    "        sharedIngredientsCount AS matchCount, " +
    "        sharedIngredients AS matchedIngredients " +
    "ORDER BY matchCount DESC, other.title ASC " +
    "LIMIT 3")
```

```
List<RecipeSuggestionDTO> findSimilarRecipes(String recipeId);
```

The query recommends similar recipes by exploiting ingredient-based relationships, ranking results according to the number of shared ingredients with the selected recipe.

Relevant Neo4j queries

Smart Fridge Recipe Suggestions

```
@Query("MATCH (i:Ingredient)-[:USED_IN]->(r:Recipe) " +  
      "WHERE i.name IN $myIngredients " +  
      "WITH r, count(i) AS matchCount, collect(i.name) AS matchedIngredients " +  
      "WHERE matchCount >= 3 " +  
      "MATCH (r)-[:WRITTEN_BY]->(c:Chef) " +  
      "RETURN r.mongo_id AS id, " +  
      "      r.title AS title, " +  
      "      r.imageUrl AS imageUrl, " +  
      "      c.name AS chefName, " +  
      "      c.surname AS chefSurname, " +  
      "      c.mongo_id AS chefId, " +  
      "      matchCount, " +  
      "      matchedIngredients " +  
      "ORDER BY matchCount DESC")
```

```
List<RecipeSuggestionDTO> findRecipesByIngredients(List<String> myIngredients);
```

The query suggests recipes that uses at least three ingredients of the foodie's smart fridge, prioritizing those with the highest number of ingredient matches.

Neo4j Indexes

All Neo4j queries start from a specific node identified by a unique property. Without indexes, every lookup performs a full AllNodesScan (~18,859 nodes).

Performance Tests

- Execution of most relevant queries
- Tests repeated 5 times clearing query cache between each request

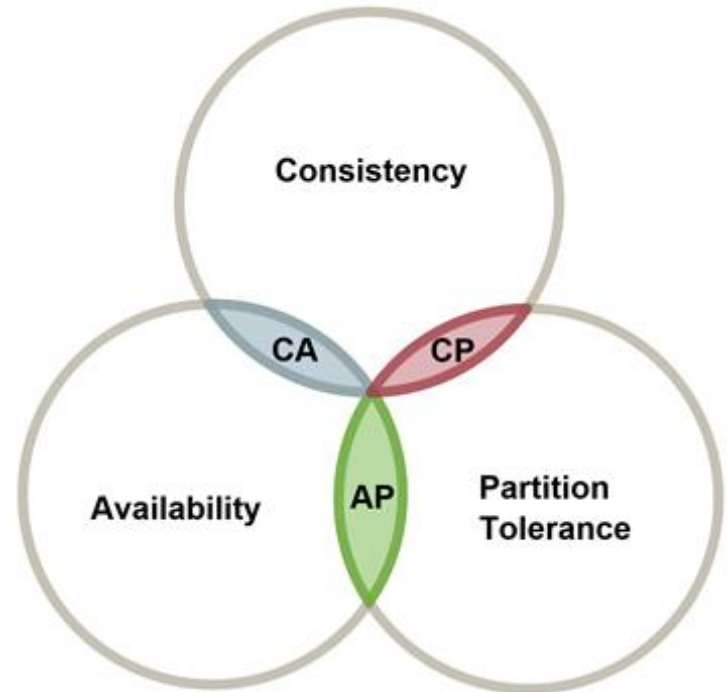
All indexes transform AllNodesScan (full graph scan) into NodeIndexSeek (direct access), eliminating the expensive initial lookup phase.

Query	Index	DB Hits (before → after)	Time
Smart Fridge Suggestion	ingredient_name_idx	51,733 → 11,080	53 ms → 28 ms
Find Similar Chef	chef_mongo_id_idx	661,062 → 623,252	206 ms → 191 ms
Similar Recipes Suggestion	recipe_mongo_idx	79,948 → 39,296	100 ms → 62 ms

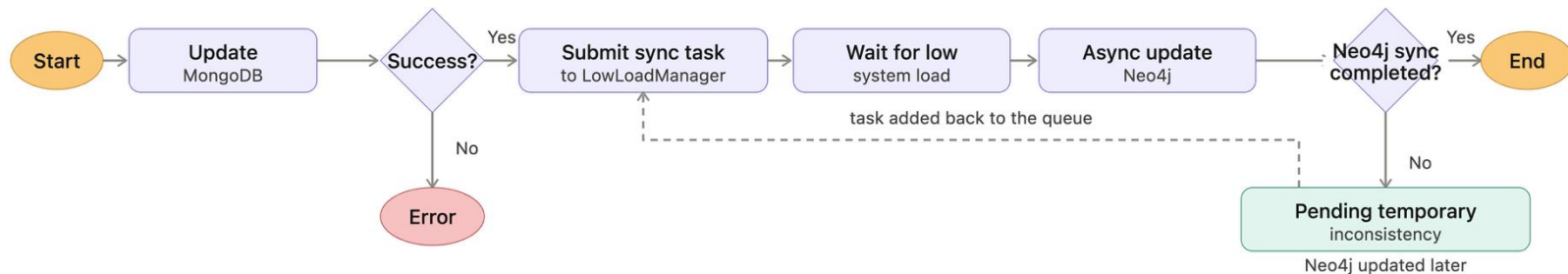
CAP – theorem consideration

The two selected properties from the CAP theorem for the system were **Availability** and **Partition Tolerance (AP)**

- Ensures that the system remains accessible and responsive to users.
- Allows the system to continue operating despite network partitions.



Handling Intra-DB Consistency



Redis cache rules

- **Add ingredient**
Invalidate suggestion cache
- **Lazy eviction (read-repair)**
Stale recipe persists in Redis after deletion from MongoDB/Neo4j → on access, "not found" triggers on-the-fly removal
- **TTL expiration for orphaned data**
Deleted Foodie profile → no synchronous delete to Redis; 15-day TTL, refreshed on interaction
- **Remove ingredient**
Update Redis Smart Fridge directly (no Neo4j traversal needed; ingredient simply removed from cached set)

Swagger UI REST APIs documentation

Overview

Admin

Admin Endpoints for platform administration (Requires ADMIN role)			^
POST	/api/admin/approveChef/{username}	Approve a chef registration	🔒 ▼
POST	/api/admin/approve/{id}	Approve a pending recipe	🔒 ▼
GET	/api/admin/showRecipes/{pageNumber}	Get pending recipes	🔒 ▼
GET	/api/admin/showChefs/{pageNumber}	Get pending chef requests	🔒 ▼
GET	/api/admin/monthlyFoodies/{year}	Get monthly user registration analytics	🔒 ▼
GET	/api/admin/details/recipe/{recipeId}	Get recipe details	🔒 ▼
GET	/api/admin/details/chef/{username}	Get chef details	🔒 ▼
GET	/api/admin/chefsRanking	Get bayesian chef ranking	🔒 ▼
GET	/api/admin/categoryTrends	Get category trend analytics	🔒 ▼
DELETE	/api/admin/discardChef/{username}	Decline a chef registration	🔒 ▼
DELETE	/api/admin/discard/{id}	Discard a pending recipe	🔒 ▼

Swagger UI REST APIs documentation

Recipes

Recipes Endpoints for viewing and searching recipes (Public Access)			^
GET	/api/recipes/{id}	Get recipe's details	🔒 ▼
GET	/api/recipes/search/{title}/{pageNumber}	Search recipes by title	🔒 ▼
GET	/api/recipes/homeRecipe/{pageNumber}	View newest recipes	🔒 ▼
GET	/api/recipes/chef/{chefId}/{pageNumber}	View chef's recipes	🔒 ▼
GET	/api/recipes/categoryTrend	Retrieve top recipe per category	🔒 ▼
GET	/api/recipes/category/{category}/{pageNumber}	Search by category	🔒 ▼

Swagger UI REST APIs documentation

Chef

Chef Endpoints for managing Chef profiles and their recipes

POST /api/chefs/changeInfo Change chef's information

POST /api/chefs/addNewRecipe Submit a new recipe

GET /api/chefs/showWaiting/{pageNumber} Show chef's pending recipes

GET /api/chefs/show/{pageNumber} Show published recipes

GET /api/chefs/popular/{pageNumber} Show popular recipes

GET /api/chefs/info Retrieve the chef's personal information

GET /api/chefs/details/{recipeId} Get recipe details

DELETE /api/chefs/removeWaiting/{id} Remove a pending recipe

DELETE /api/chefs/deleteRecipe/{id} Delete an already approved recipe

DELETE /api/chefs/deleteProfile Delete chef's profile

Swagger UI REST APIs documentation

Foodie

Foodie Endpoints for managing Foodie profiles and their favorite recipes

POST	/api/foodies/changeInfo	Change foodie's personal information	🔒	▼
POST	/api/foodies/addFavourite/{recipeId}	Add a recipe to favourites	🔒	▼
GET	/api/foodies/similarRecipes/{recipeId}	Show similar recipes	🔒	▼
GET	/api/foodies/recipe/{id}	Retrieve saved recipe details	🔒	▼
GET	/api/foodies/matchingChef	Search for chefs by surname	🔒	▼
GET	/api/foodies/info	Retrieve foodie's information	🔒	▼
GET	/api/foodies/getRecipe/{filter}/{pageNumber}	Retrieve favourite recipes filtered	🔒	▼
GET	/api/foodies/findSimilarChef/{chefId}	Show similar chefs	🔒	▼
DELETE	/api/foodies/removeFavourite/{recipeId}	Remove a recipe from favourites	🔒	▼
DELETE	/api/foodies/deleteProfile	Delete foodie's profile	🔒	▼

Swagger UI REST APIs documentation

Auth

Authentication Endpoints for user registration and login			^
POST	/api/auth/register/foodie	Register a new foodie	🔒 ▼
POST	/api/auth/register/chef	Register a new chef	🔒 ▼
POST	/api/auth/login	User login	🔒 ▼

Ingredients

Ingredients Endpoints for managing ingredients allowed in the service			^
GET	/api/ingredients/allowedIngredients	Allowed ingredients	🔒 ▼

Swagger UI REST APIs documentation

SmartFridge

Smart Fridge Endpoints for managing the smart fridge and getting recipe recommendations

POST	/api/fridge/add	Add ingredients	🔒	▼
GET	/api/fridge/recommendations/{pageNumber}	Get recipe recommendations	🔒	▼
GET	/api/fridge/recipe/{id}	Get recipe details	🔒	▼
GET	/api/fridge/get	Retrieve smart fridge	🔒	▼
DELETE	/api/fridge/remove	Remove ingredient	🔒	▼

ShoppingList

Shopping List Endpoints for managing the user's shopping list

POST	/api/shopping/add	Add ingredients	🔒	▼
GET	/api/shopping/suggestedIngredients	Suggest ingredients	🔒	▼
GET	/api/shopping/get	Retrieve shopping list	🔒	▼
DELETE	/api/shopping/remove	Remove ingredient	🔒	▼

Java doc documentation

Packages

Package	Description
it.unipi.MySmartRecipeBook	
it.unipi.MySmartRecipeBook.config	
it.unipi.MySmartRecipeBook.controller	
it.unipi.MySmartRecipeBook.dto	
it.unipi.MySmartRecipeBook.dto.recipe	
it.unipi.MySmartRecipeBook.dto.users	
it.unipi.MySmartRecipeBook.event	
it.unipi.MySmartRecipeBook.model.Mongo.ingredients	
it.unipi.MySmartRecipeBook.model.Mongo.recipes	
it.unipi.MySmartRecipeBook.model.Mongo.users	
it.unipi.MySmartRecipeBook.model.Neo4j	
it.unipi.MySmartRecipeBook.repository.Mongo	
it.unipi.MySmartRecipeBook.repository.Neo4j	
it.unipi.MySmartRecipeBook.security	
it.unipi.MySmartRecipeBook.security.jwt	
it.unipi.MySmartRecipeBook.service	
it.unipi.MySmartRecipeBook.utils.conversionFunctions	
it.unipi.MySmartRecipeBook.utils.parameters	
it.unipi.MySmartRecipeBook.utils.populateDB	

Live Demo with Postman

